

ARCHITECTURE

IN-MEMORY ACCOUNT LEDGER CORE

Decisions, trade-offs and production considerations arising from this implementation. Everything below refers to code in this repository; the performance figures are measured, not estimated.

1. Append-only at scale

What breaks first is not volume

I expected transaction volume to be the constraint. It is not. Measured on this engine, one axis at a time (wall clock, no instrumentation overhead):

Axis	From → to	Time	Factor
Events (6 days, 2 accounts)	30 → 3,000	0.003s → 0.14s	~50× for 100× volume
Window length (10 events/day)	60 → 180 days	1.53s → 38.51s	25× for 3× the days
Accounts (30 days, 300 events)	2 → 200	0.21s → 9.18s	~44× for 100× accounts

100× the transaction volume costs a seventh of a second. **Window length is very nearly cubic** — the measured exponent is 2.7. Tripling the window multiplies the work twenty-five-fold. The ledger does not fall over because a lot happens; it falls over because time passes.

The cause is that `close_day` re-derives all of history on every close. Both sweeps iterate `range(first_day, today + 1)`, and each iteration calls `Book.closing_balance`, which is a full scan of the entry log. Summed across the window that is $O(N^2 \cdot E)$, and E itself grows with N , giving the cubic. `AppendOnlyLog.__iter__` compounds it with a constant factor by returning `iter(tuple(self._items))` — it allocates a copy of the whole log on every scan.

Where unbounded state accumulates

The entry log grows with volume. That is append-only working as intended and is not the problem.

The accrual journal is the problem. It grows with `accounts × days`, decoupled from transaction volume entirely. `accrue_interest` appends a record for every account for every day the computed accrual differs from the running net — so an account that transacts once and then sits idle keeps appending, one record per day, forever. Measured: a single credit followed by 40 idle days produces **40 accrual records**. Backdating adds a second dimension on top, one revision per affected day per account.

The only thing that stops it is the rounding dust threshold: below AED 12.50 the accrual rounds to zero, matches the running net, and nothing is appended. The journal therefore grows for exactly the accounts worth having — the same test at AED 12.49 produces zero records — which is the wrong way round.

At 200 accounts over 30 days on a fixed 300 events:

```
entries: 1,343
accruals: 2,363    <- the derived journal is larger than the log it derives from
```

That inversion is the signal. A projection that outgrows its source is storage nobody budgeted for, and it is invisible at six days and two accounts.

The cheapest structural change

A back-value window constant W , bounding both sweeps to `range(max(first_day, today - W), today + 1)`. Two `range()` bounds and one policy field.

Prototyped as `Windowed(Engine)`, overriding nothing but those bounds, $W = 5$:

Window length	Unbounded	Windowed	Speedup
30 days	0.21s	0.10s	2×
120 days	11.27s	1.59s	7×
180 days	38.25s	3.55s	11×

The speedup grows with the window because the change removes a whole factor of N — quadratic becomes linear in window length. That is the fix.

It does not fix the storage problem, and I initially wrote that it did. The first comparison showed accrual counts identical between the two engines (146/146, 251/251, 728/728), because that workload only backdated within five days and the window therefore had nothing to suppress. Re-running with deeper backdating shows what the window actually buys, on 2 accounts over 60 days:

Backdate depth	Unbounded accruals	Windowed ($W=5$)	Reduction
5 days	334	334	0%
30 days	1,005	438	56%
55 days	1,257	450	64%

So the window caps the *revision fan-out* — substantially, once backdating exceeds it — but it cannot touch the baseline `accounts × days` growth, which is the floor those windowed figures converge on (~420–450 against a 120-record minimum for 2 accounts × 60 days). Bounding storage needs a second, separate change.

In the order I would do them:

1. **Compact the accrual journal at capitalisation.** Net the revisions into one record per `(account, period)` and archive the detail. This is what actually bounds the storage growth the window leaves untouched.
2. **Materialise a running balance per `(account, value_date)`**, invalidated forward from the value date of any backdated arrival. Turns each O(E) scan into a lookup plus a short delta. A cache over the existing model, not a different model — the projection stays the source of truth.

Before either, `AppendOnlyLog.__iter__` should stop copying the log on every scan — a fixable constant that currently flatters nothing but the excuse.

2. Value-dated entries in production

A back-valued entry does not add a row. It changes what already-published history *was*. Everything below follows from that one property.

Operational surface

- **Statements are retroactively wrong.** This implementation restates Day 2 from 250.00 to 225.00 after the fact. A statement cycle that has already been issued needs re-issue or a correction notice, and a support function able to explain two different true answers to the same question.
- **Fees land in days the customer has already seen.** Two of the three overdraft fees here are value-dated into days that closed in credit and were reported as such. A customer is being charged, after the fact, for a day they were told was fine.
- **Interest is recomputed for closed days**, producing back-valued adjustments that must be disclosed rather than silently netted.
- **Accounting period close.** The general ledger is shut at month end. This implementation has no concept of a closed period and will happily post into one.

Regulatory surface

- **Prior-period adjustment and disclosure** obligations (IAS 8) once a restatement is material.
- **Reports already filed move.** Regulatory returns are computed on historical positions, so a backdated entry changes a figure already submitted.
- **Conduct risk.** Retroactively charging an overdraft fee for a day the customer was told they were in credit is precisely the fact pattern that draws consumer-protection enforcement. This implementation does exactly that, three times.
- **Retention.** Both the original and restated views have to be kept. The `booked_day / value_date` pair is the one thing this design already gets right here: it can always answer *what did we believe, and when*.

One control before go-live

A hard back-value window enforced at the ledger boundary, with anything older than the window requiring a dual-authorized prior-period adjustment booked in the current period.

Why this one, over a reconciliation report or a restatement alert:

- It is **preventive**. Detective controls tell you a reported period was rewritten after it has been rewritten.
- It **bounds blast radius**: no entry can silently reach into a period that has been stated, closed or reported.
- It **forces the exception into an attributable, four-eyes path** — the artefact an auditor asks for is not "can this happen" but "who approved it".
- The enforcement point already exists. `Event.__post_init__` rejects forward value-dating today; the backward bound is the same validation.

`W` should be set to the operational correction cycle (commonly ~5 business days), with a hard stop at accounting period close rather than a rolling day count — the risk is defined by what has been *reported*, not by elapsed time.

3. Authorization lifecycle

In this model, there is one exit

Stated plainly, because it is the finding: `_apply_settlement` is the only code path that writes a `RELEASED` hold record.

An authorization can also end by never beginning — **declined at request**, where the available-balance test fails, no hold is placed, nothing posts, and a `DECLINED` decision is recorded. And a settlement need not match: under-settlement releases the hold **in full** rather than stranding the shortfall, and over-settlement is permitted and posts the actual amount. Both are still that one exit.

That is the complete list. There is no expiry, no void, no cancellation. **An approved, unsettled authorization in this model is immortal** — it suppresses available balance forever. Auth-B would have demonstrated this had it been approved; it is declined, so the defect never shows in the output.

`HoldRecord` already carries `action` and `memo`, and `HoldState` already exposes `release_reason`. The data model can express every ending below today. Only the lifecycle logic is missing.

What must be able to end an authorization, and how

Ending	Real-world scenario	Mandated behaviour
Expiry	Merchant never settles: abandoned checkout, cancelled booking, failed delivery.	Auto-release at an expiry that varies by scheme and merchant category, travel and vehicle rental running materially longer than retail. Released by a scheduled sweep as an appended record with reason <code>EXPIRED</code> — never by mutation, never by deletion.
Late settlement after expiry	Merchant settles after the hold is gone. Routine, not exceptional.	Accept — the purchase happened. Post as a force-post with no hold to consume, re-run the available-balance test for reporting only, and raise an exception item. Never allow an unexplained negative available balance to appear with no record of why.
Authorization reversal / void	Merchant cancels before settling: order cancelled, item out of stock, tip adjusted down.	Release with reason <code>REVERSED</code> , referencing the reversal message. Kept distinct from <code>EXPIRED</code> in reporting — the two say different things about merchant behaviour and feed different remediation.
Partial reversal / decrement	Order partially fulfilled.	Append an adjustment reducing the held amount. Do not edit the hold, and do not release-and-replace, which severs the linkage to the original authorization.
Incremental authorization	Hotel extends a stay; fuel final amount exceeds the pre-auth.	New <code>PLACED</code> record under the same auth id. Re-run the available-balance test on the increment alone — a failure declines the increment without disturbing the original hold.
Issuer-initiated cancellation	Card blocked for fraud; account frozen.	Release the hold so the customer's available balance is restored, but mark the authorization so any later settlement routes to dispute instead of posting.
Account closure with holds outstanding	Customer closes an account with a pending hold.	Block closure while holds are active. Forced closure requires write-off under dual authorisation with the residual liability recognised — never a silent drop.

Two of these are not expressible here at all without a prior change: expiry and any time-based release need **real timestamps**, and this model has only day ordinals (section 4).

4. What I cut, and why

Ranked by the severity of the risk deferred, not by size.

Cut	Why it was cut	Production risk deferred
Single-entry, not double-entry	<code>Entry</code> has one <code>account_id</code> and a signed amount. No contra leg, no transaction grouping forcing legs to sum to zero. Adding one is a structural change beyond the brief.	No trial balance. A lost, duplicated or misrouted posting is structurally undetectable — nothing in the system can prove the books balance. The first thing I would fix.
No ingestion idempotency	Only duplicate <code>auth_id</code> and double-reversal are guarded. A replayed <code>event_id</code> posts twice.	Any at-least-once delivery — every real queue — double-posts money . Cheap to fix, severe if not.
Policy is not effective-dated	<code>Policy</code> is one global object.	The design recomputes history from the log on every close, so changing the fee from 25 to 30 makes a replay of the <i>same events</i> produce <i>different history</i> . Reproducible history is the one guarantee event sourcing exists to provide, and this forfeits it. Policy must be versioned by effective date.
No authorization expiry	No period specified; inventing one changes results arbitrarily.	Immortal holds permanently suppressing available balance — section 3.
Day ordinals, no timestamps	The brief gives days, no clock.	No intraday ordering beyond stream position, no cut-over policy, and time-based release is inexpressible.
No concurrency control	Single-threaded replay.	Two settlements racing one authorization both observe it active. Needs optimistic versioning on the hold.
No day-count convention	"0.04% per day", flat.	No ACT/365 vs ACT/360, no leap year, no business-day calendar. Interest that does not match the product's own terms is a dispute and a remediation.
No FX or multi-currency accounts	Out of scope.	<code>Money</code> refuses cross-currency arithmetic, which is the right default, but there is no conversion path at all.
Fee reversal on restatement	Deliberate; the decision is a commercial one.	Documented at length in <code>REJECTED.md</code> and kept visible by the failing test in <code>tests/test_known_design_gap.py</code> .

Six further cuts are each a single fact, so they are listed rather than tabulated: **no closed accounting period** and **no back-value window** (section 2), **no snapshot over the O(n) projections** (section 1), **no account lifecycle**, leaving section 3's closure case nowhere to live, **no fee cap** — three fees on one account in one window — and **no persistence**, which the brief mandated and which leaves the ledger as process state.

The three I would fix before anything else: ingestion idempotency (cheapest, and it loses money), contra legs (nothing else can prove correctness without them), and effective-dated policy (without it, replay is not reproducible, and replay is the entire design).